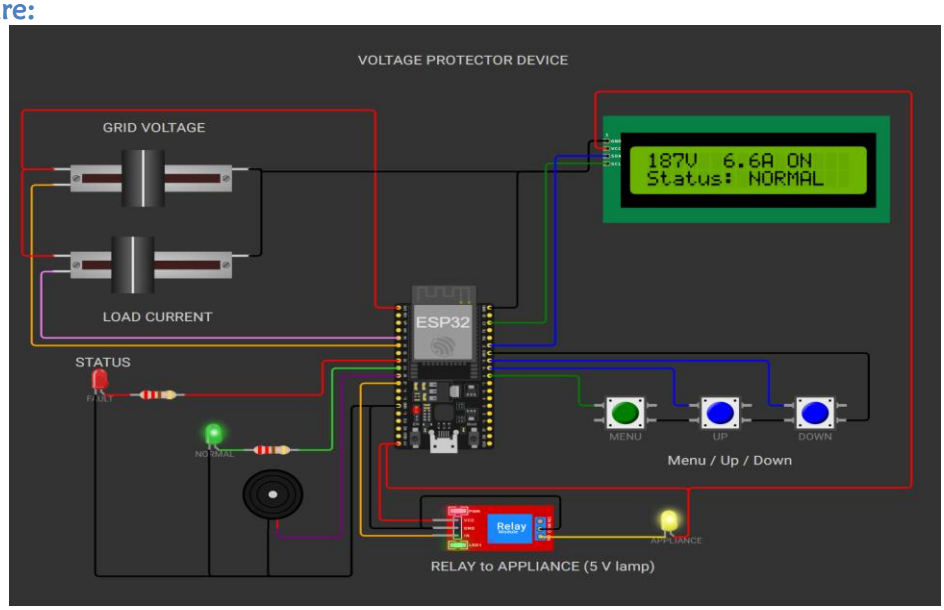
Voltage Protector using ESP32

Introduction:

Appliances can get damaged when the line voltage goes too high or too low, or when they draw too much current. The project concept is to save the apliences from high voltage surge and over current. An ESP32 checks the voltage and current all the time and cuts the appliance off with a relay when something goes wrong.

Architecture:



Block Diagram

The project is based on the Arduino platform with an ESP32. Two potentiometers are used in place of the voltage and current sensors. GPIO35 reads the voltage (0-440V) and GPIO34 reads the current (0-20A). The relay is on GPIO27 and switches a 5V lamp that works as the appliance. For the user side there is a 16x2 I2C LCD (SDA 21, SCL 22), three buttons MENU (5), UP (18) and DOWN (19), a green LED (25), a red LED (33) and a buzzer (26). The PC is connected by USB serial at 115200 baud, and I made a small python GUI to see the values and change the settings. The code is modular, I split it into settings, protection, commands, buttons and display files.

Working:

Every 5 ms the ESP32 takes 16 samples from each pot and uses the average. The device works in three states. In NORMAL the relay is on. If the voltage goes over or under the limit it turns off and goes to WAITING, and if the current goes over the limit it goes to LOCKED. In WAITING it turns on again only when the voltage stays ok for the reconnect delay. I used 5V hysteresis so the relay does not chatter near the limit. The device also starts in WAITING after power up. In LOCKED the relay stays off until the user presses the UP button or sends RESET from the PC. Default settings are 250V, 180V, 10A and 5s. Every new value is range checked, and OV must be at least 30V above UV. The settings are saved in EEPROM with a magic byte, so they stay after power off.

Results:

The Code is working fine on simulation as well as in the real hardware, In the real hardware I did not test with Mains AC High voltage but I keep the Voltage sensor and current sensor for further development, for testing I simulate the voltage and current using 2 Potentiometer.

Conclusion:

I will add IoT features in the future, as I choose esp32 as the Microcontroller. The device works as asked. For a real product I will use ZMPT101B and a CT sensor with true RMS measurement, and a MOV for the fast spikes.